

Problem 2

Overview and Setup

We can sort all $3n$ points of the triangles by their x-coordinates and use this information to set up a queue, Q , that will store the events as x-coordinates in the order we will examine them.

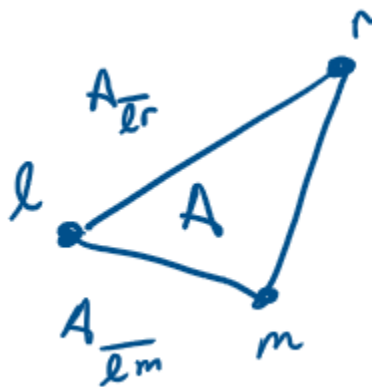
While we are conducting the plane sweep, we will maintain a BST that tracks all the edges of the triangles that are being evaluated. Edges contain the information needed to calculate their y-coordinate at a given event, Q_i , as well as their descriptive end points. I assume Edge $A_{LR} = \text{Edge } A_{RL}$ and can be identified in code as the same. I also assume vertices give information on their respective edges, as well as the island they belong to.

Additionally, we will keep track of connected islands using a Union-Find structure, `Connected`. This allows us to see if islands are connected in groups, if the groups are disjoint, and allows us to easily combine connected groups.

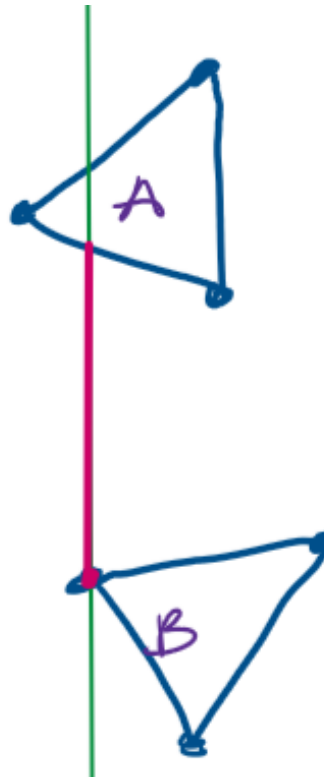
Finally, we'll keep track of bridges in a list, `Bridges`.

Sweep Algorithm

Starting at the first event, Q_1 , we will examine the leftmost point of the first island, let's call it Island A. The leftmost point starts two new edges, we can call A_{LR} and A_{LM} .



We will insert the two edges into our BST with A_{LR} being above A_{LM} , this could be checked by assessing slope since both lines have the same 'intercept'. We now move onto the next event, Q_2 .



At any subsequent Q_i , traverse the BST to find the island edges that are neighbors to where our corresponding point, P_i , would lie. We will call these edges A and B for above and below, respectively, but they may not be in the case of islands being next to each other.

I will then check to see if the current island of P_i is currently connected to the island of edge A using the Union-Find structure, Connected. If not, I will connect P_i to edge A using a vertical or horizontal edge. I will also record this bridge in Bridges. Once I connect the edges, I will also merge the islands in Connected.

From here I need to assess if P_i is a leftmost, middle, or rightmost vertex. This could be determined by assessing how many edges come out of P_i .

If P_i is a leftmost vertex, insert P_{LR} and P_{LM} into the BST.

If P_i is a middle vertex, insert P_{MR} into the BST and find and remove P_{LM} , since it will no longer be used.

If P_i is a rightmost vertex, remove both P_{MR} and P_{LR} .

However, there is a important caveat. We will never remove a pair of rightmost edges if they would leave the BST empty. This is important because if islands lie left-right to each other, then they would not be able to be connected. The BST would never be empty unless the edges for the rightmost point are removed. Because this is true, we will keep them inside the BST and allow a potential future point to be connected.

We now move onto the next event, Q_{i+1} , and repeat until we are done iterating over Q and return Bridges.

Time Complexity

n represents the number of islands.

Setup:

The initial construction of Q is $O(3n \log(3n))$ for sorting of all points and construction of the queue.

Sweep:

The algorithm iterates over a total of $3n$ events. At each event, two BST searches are performed, at $O(\log(3n))$ each. Union operations are constant time. Addition into the list Bridges is constant time (Bridges could also be a stack, since we only add to it). Final operations during the leftmost-middle-rightmost check involve at most two insertions and removals on a BST, giving $O(\log(3n))$ as well.

Overall:

Setup: $O(3n \log(3n))$

Sweep: $O(2 \log(3n)) + O(1) + O(2\log(3n))$

Total:

$O(n \log(n))$

Problem 3

Overview and Setup

We can describe each circle, C_i , as its leftmost ($C_{i,L}$) and rightmost ($C_{i,R}$) points. Additionally, we can split the circle into two arcs, which will be necessary for implementing a plane sweep. We can split the circle into upper and lower arcs, U_i and L_i , respectively.

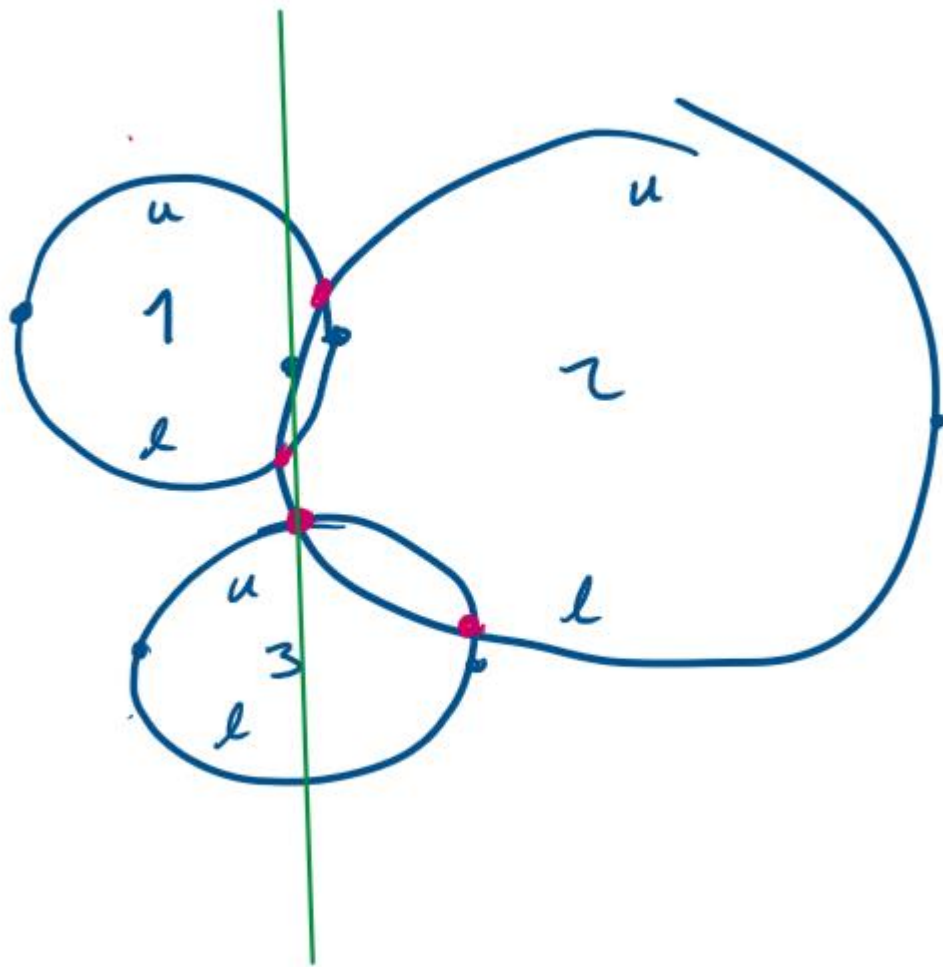
So, we no longer have C_i , but rather U_i and L_i , which have the same leftmost and rightmost points.

To determine the order of the sweep, we will sort all points on the plane by their x-coordinates and we will store this order in a queue, Q , stored as a min-heap. We will also distinguish between left, right, and intersection points as events in the queue.

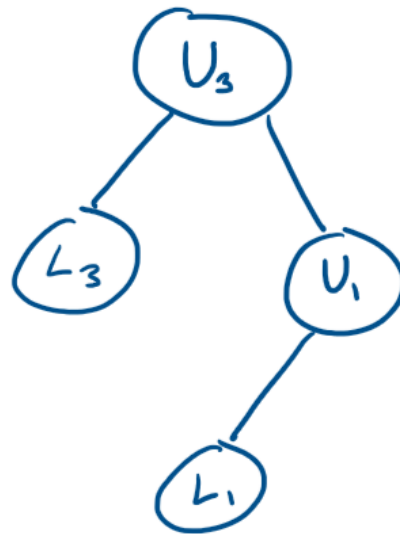
Sweep Algorithm

As we sweep, we will insert the arcs into a BST, T , which is sorted by y-coordinates, with upper arcs being considered 'higher' than their matching lower counterparts.

With any given event in Q , Q_i , we will insert the arcs that correspond to that event. For example, at Q_i , we will insert the upper first into T . But importantly, as we insert into T , we must evaluate the current position of the arcs as we compare during insertion.

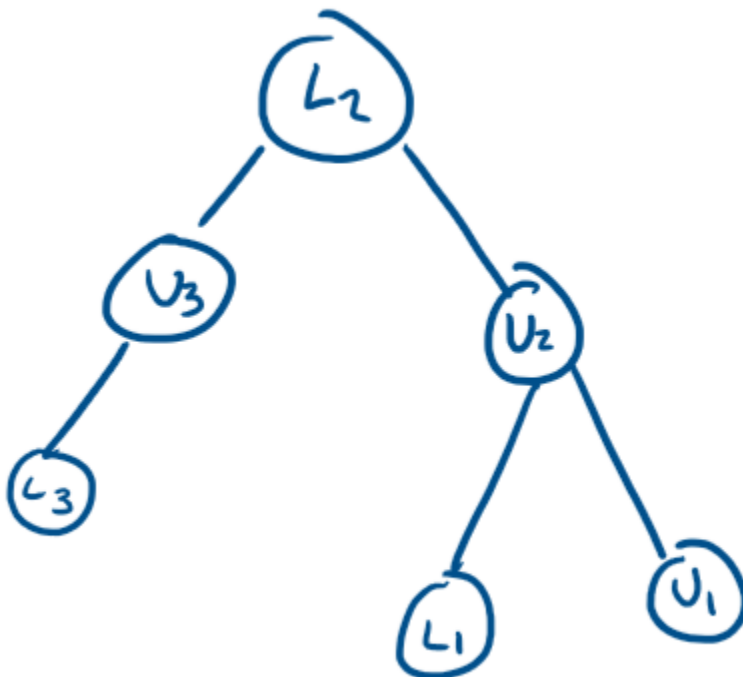


For example, at the event corresponding to $C_{2,l}$, we will evaluate the T , to determine where to insert U_2 and L_2 . At this point, the BST should look like this because of prior events in Q .



Leftmost points:

As we insert U_2 and L_2 , starting with U_2 , we will calculate and evaluate that U_3 is currently higher than U_2 . So, we go lower into the tree and assess the next point U_1 and U_2 . We evaluate that U_2 is below U_1 and above L_1 and, crucially, we insert U_2 and L_2 as a pair so that they are neighbors to each other. Like so:



Once inserted, we will check for intersections between the lower neighbors of the L_i arc with L_i , and upper neighbor of the U_i arc with U_i . If intersections appear later, we add their point coordinates to Q . Here we can see that there will be intersections at both evaluations for U_1 & U_2 and L_1 & L_2 .

At intersections, we will swap arcs that are being intersected. For example, at the intersection of U_2 and U_1 we must swap because going forwards, they will change in vertical positions. After the swap is completed, we check the neighbors once again to determine if there will be any intersections, mirroring the actions done at leftmost points.

At rightmost points, for a given Q_i look at the arc immediately above U_i (using T) and the arc immediately below corresponding lower arc, L_i . We will call the above arc, A , and the below arc, B . We will remove U_i and L_i and connect A and B in T . Lastly, we check for a future intersection between A and B . If it exists, we add it into Q .

Time Complexity

Setup: Initial sorting is performed on all $2n$ leftmost and rightmost points from the circles to create the event queue, Q . This costs $O(n \log(n))$.

Sweep: At a higher level, there are $2n + k$ events, $2n$ for all leftmost and rightmost points and k intersections.

At all $(2n+k)$ events, we perform a $\log(n)$ traversal and then constant time operations for all three cases (leftmost, rightmost, and intersection). For insertion into the queue, on the finding of insertions, this is another $\log(n)$ operation. For sweeping, we get $O(2n \log(n) + k \log(n))$.

Overall:

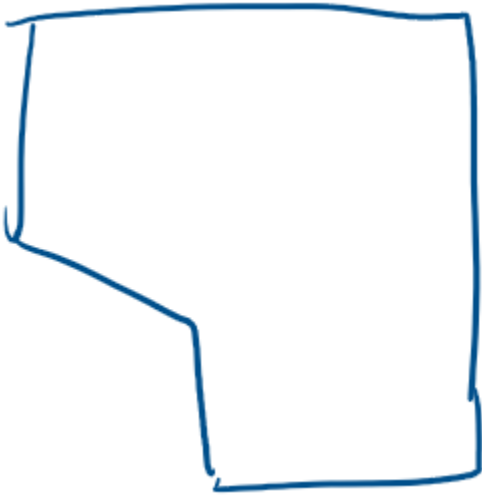
$O(n \log(n) + 2n \log(n) + k \log(n)) \rightarrow$

$O((3n+k) \log(n)) \rightarrow$

$O((n+k) \log(n))$

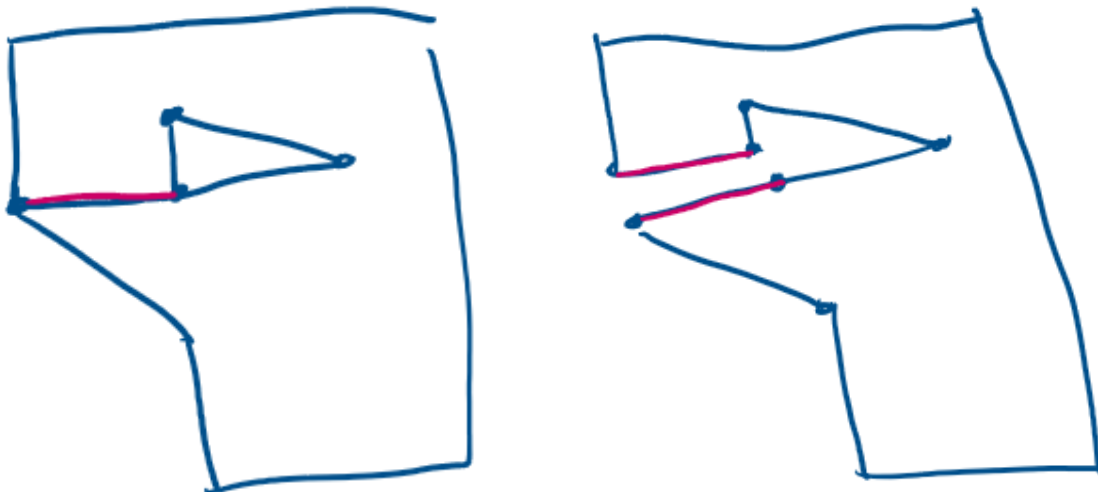
Problem 4

Consider this polygon, P , with n vertices.



We have already established that the number of triangles that can be constructed is $n-2$.

Given a hole in P , we can make any singular cut from a hole to another vertex on P to transform its representation from a polygon with a hole to a simple polygon, P' .



Observe that the separation we have made has 'added' 2 more points for the singular hole. Because P' now represents a simple polygon, we can apply the same proof for

simple polygons, but with 2 additional vertices for each additional hole. Thus, our final formula is $(n + 2k) - 2$.

Problem 5

Brief description of algorithm (lack of time):

Assuming the polygon has already been triangulated using the $O(n \log n)$ algorithm discussed in class, we can create a dual graph by iterating over the triangles and creating a node for any triangle that exists and connecting the node to another if they share an internal diagonal.

We can root the tree, T , at any point.

Once rooted, the tree is not balanced, but we can traverse the tree and assign 'sizes' describing the number of nodes contained by the node's subtrees. Using a traversal in n time, we can traverse the tree starting from the minimum, accurately tracking the sizes as we traverse until we reach a size as close to $n-2/2$. If we overshoot it, we can return the previous node and its diagonal, if not, return that node we are on.

$O(n \log n)$ for triangulation

$O(n)$ for graph construction

$O(n)$ for initial traversal and size assignments.

$O(n)$ for finding the diagonal

Total: $O(n \log n)$

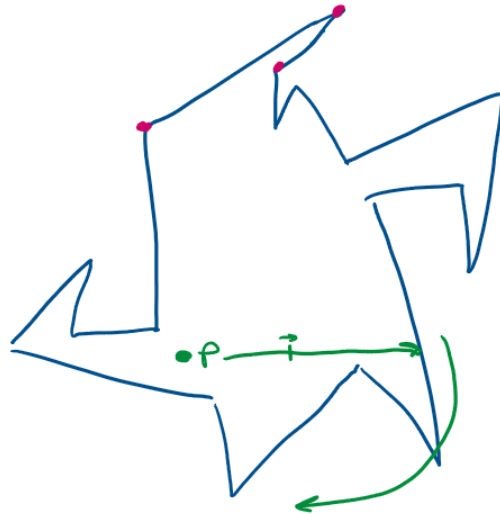
Problem 6

Note: I saw the Joe and Simpson Algorithm as a solution, but did not understand it, so I am writing this $O(n \log(n))$ solution instead.

Overview and Setup

We will calculate the polar angles of all vertices relative to point P inside the polygon. We will store these in order of angle inside a queue, Q . We will store the edges created by all vertices in a BST sorted by distance to the internal point P .

Prior to sweeping, we will initialize the BST by iterating through all the edges of the polygon to determine which would be intersected by an initial horizontal ray.



We will store any edge that intersects this initial ray in our BST.

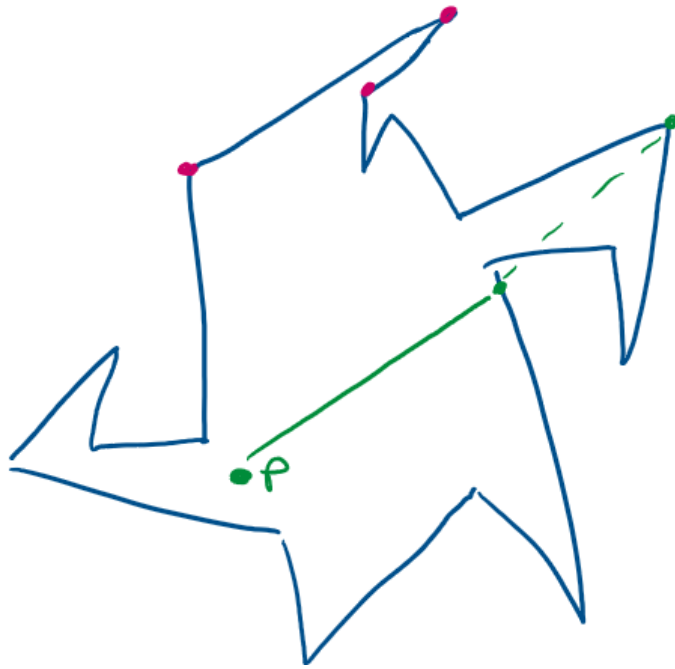
We will store the vertices that describe the final region in the stack Visible.

Sweep Algorithm

We start at Q_i , at the point closest to the polar angle, 0. From here, we add any edges connected to vertex V_i into our BST. We will also remove any edge that ends at V . This can be found in $\log(n)$ time due to the nature of a BST.

Observe that the closest edge (or the minimum of the BST) describes the visible boundary in this case.

Using this closest edge, we will calculate the point along the closest edge that aligns with our polar angle. EX:



We will add this calculated vertex to Visible.

Time Complexity

Setup: $O(n \log n + n) \rightarrow O(n \log(n))$

Sweep: $O(n \log n)$

Overall:

$O(n \log(n))$